



16

ENCODING AND ENCRYPTION



Encoding is the act of converting data into a new format.

It's used for tasks such as efficiently transporting data, ensuring interoperability between protocols or applications, and compressing and storing data. Malware authors also use encoding to obfuscate data and code that shouldn't be seen by analysts or host and network defenses. *Encryption*, which shares similarities with encoding, is a way to protect

sensitive data in transit or at rest. Malware can use encryption for many reasons, including obfuscating sensitive data in memory such as command and control (C2) information. In addition to defense evasion, malware often uses encoding and encryption to impede analysis, especially static code analysis or analysis of network traffic.

Specifically, malware utilizes encoding and encryption algorithms for the following reasons:

- To protect code and data in its files that reside on disk, in order to evade endpoint defenses such as anti-malware software. This often involves a technique called packing that will be discussed at length in [Chapter 17](#).
- To protect code and data in memory (such as C2 addresses, keys, or sensitive strings) from endpoint defense software and analysts.
- To protect its data on the network layer while in transit, for example, by encrypting data before transferring it to a C2 infrastructure.
- To hamper reverse engineering and analysis efforts. If a malware analyst must first decode or decrypt parts of the malware's code, it can

slow and frustrate the analysis process.

In this chapter, you’ll see some encoding and encryption techniques used by malware and learn a few practical tips to overcome them when analyzing it.

Basic Encoding

One of the most widely used forms of encoding is *Base64*, which was originally designed for data transfer and interoperability between various protocols. When a string is run through a Base64 encoding algorithm, the string is fed into the algorithm as binary data and broken down into 6-bit blocks. Each block is then translated (encoded) to ASCII format, using a total of 64 different characters (26 lowercase characters, 26 uppercase characters, 10 digits, and the / and + characters). This 64-character set, shown in **Table 16-1**, is where Base64 gets its name. The table shows the character (Char), decimal value (Dec), and binary value (Bin) of each of the Base64 characters.

Table 16-1: The Base64 Character Set

Char	Dec	Bin	Char	Dec	Bin	Char	Dec	Bin
A	0	0	L	11	1011	W	22	10110
B	1	1	M	12	1100	X	23	10111
C	2	10	N	13	1101	Y	24	11000
D	3	11	O	14	1110	Z	25	11001
E	4	100	P	15	1111	a	26	11010
F	5	101	Q	16	10000	b	27	11011
G	6	110	R	17	10001	c	28	11100
H	7	111	S	18	10010	d	29	11101
I	8	1000	T	19	10011	e	30	11110

Char	Dec	Bin	Char	Dec	Bin	Char	Dec	Bin
J	9	1001	U	20	10100	f	31	11111
K	10	1010	V	21	10101	g	32	100000
h	33	100001	s	44	101100	3	55	110111
i	34	100010	t	45	101101	4	56	111000
j	35	100011	u	46	101110	5	57	111001
k	36	100100	v	47	101111	6	58	111010
l	37	100101	w	48	110000	7	59	111011
m	38	100110	x	49	110001	8	60	111100
n	39	100111	y	50	110010	9	61	111101
o	40	101000	z	51	110011	+	62	111110
p	41	101001	0	52	110100	/	63	111111
q	42	101010	1	53	110101			
r	43	101011	2	54	110110	=	(padding)	

Notice the equal sign (=), which is used as padding and thus doesn't count as one of the 64 characters. When analyzing malware code, if you spot a string of characters followed by one or more equal signs, an alert should sound in your head that you might be dealing with some variation of Base64-encoded data. Note that this padding character isn't always present.

The Base64 character set is often hardcoded in malware, like so:

```
character_set = "ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz0123456789+/"
```

If you spot something in code that looks like this index string, you may assume the malware is using a variation of Base64 encoding. We'll return to this in a moment.

To better understand what Base64 encoding looks like in practice, let's look at an example, using the string `evil` as input. The ASCII string `evil`, when converted to binary, reads as follows:

e	v	i	l
01100101	01110110	01101001	01101100

If we run the `evil` ASCII string through a Base64 encoding algorithm, the ASCII is converted to binary, this binary string is broken into 6-bit blocks, and each block is matched against one of the characters in the Base64 character set. For the `evil` string, these blocks and values would look like this:

Block 1	Block 2	Block 3	Block 4	Block 5	Block 6
011001	010111	011001	101001	011011	00
Z	X	Z	p	b	A

The Base64 string output here is `ZXZpbA`. However, most implementations of Base64 group output in sets of six characters and, if there's an odd number of characters at the end of the encoded string, add padding as needed. In this case, Block 6 isn't complete, so `==` is added as padding and the end result is `ZXZpbA==`.

A full discussion of Base64 encoding is outside the scope of this book. For a more complete explanation of the technical details of this algorithm, see <https://en.wikipedia.org/wiki/Base64>.

While Base64 has plenty of legitimate use cases, malware can also use it as a quick-and-dirty way to achieve a more evil purpose: obfuscating data. Base64 (and other encoding algorithms like it) can be used to hide malicious behaviors or strings in the file itself or in memory. The advantage of using encoding algorithms like Base64 is that they're incredibly simple to implement. The disadvantage is that they're also incredibly easy

to “break”; you can simply decode the target data in the same way it was encoded. Base64 can be decoded using many different methods and tools, one of which is CyberChef (<https://github.com/gchq/CyberChef>), which we’ll touch on in a moment.

There are many great tools that can help you identify Base64 usage in malware. For example, *base64dump.py* (<https://github.com/DidierStevens/DidierStevensSuite/blob/master/base64dump.py>) is a Python script by Didier Stevens that scans an input file and attempts to find encoded data within it. It will list any encoded strings it finds and display their decoded value, allowing the analyst to quickly identify suspicious encoded data. **Figure 16-1** shows the output of this tool.

```
remnux@remnux:~/Documents$ base64dump.py evil.bin
```

ID	Size	Encoded	Decoded	md5 decoded
1:	16	xxxxxxxxxxxxxxxx	..q..q..q..q	8ef1e9f18878c82c70238450b683c1c7
2:	4	xjhf	.8	d20d18b50a544e11a506e5583e95b4cc
3:	4	Zjrf	f:.	e8e0bef1b9bee326ce67d13be993d82d
4:	40	aG9tZXNzaGVhbnR0	homesshealth.inf	a67bd3646d3c48d341589a7f25633813
5:	4	xxXP	..0	f1cdec9b91130e582f05e698dbc07462
6:	4	Xjrf	^:.	2b4ef6c5176ec9cfc9315fd19a0dfdbc
7:	4	Xjif	^8.	7e556efb1068e05ec560030322096de9
8:	4	Xjif	^8.	7e556efb1068e05ec560030322096de9
9:	8	xxxxxxx	..q..q	12fd2ab650e160fb7a36a16295137f77

Figure 16-1: Viewing Base64-encoded strings from a binary in *base64dump.py*

Notice the possible decoded string in line 4 (*homesshealth.inf*). You can see part of the encoded value as well. To fully assess this decoded string, you can *dump* it (extract it and save to disk) by using the *-d* (dump) and *-s* (section) switches in *base64dump.py*, specifying the section number of interest (in this case, 4):

```
> base64dump.py evil.bin -s 4 -d
```

Note that malware authors can easily modify standard Base64 encoding to inhibit reversing of their data. Simply modifying the Base64 character set will drastically change its output. Consider, for example, the following modified index string:

```
abcdABCDEFGHijklmnopQRSTUVWXYZefghijklmnopqrstuvwxyz0123456789+/
```

Compare this with the original Base64 character set mentioned previously:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-
```

Notice the difference? In the first set, the characters `abcd` were moved to the beginning of the character set, before `ABCD`. Simply moving `abcd` to the beginning of this index will drastically change the output of Base64 encoding, and this is a very easy change to make. To detect these types of changes, you can look for the index string in code, as mentioned previously. Quite often, this Base64 index will be clearly visible in a malware executable's code.

Finally, keep in mind that there are other encodings you might encounter when analyzing malware (such as Base32, Base85, uuencode, and yEnc), but these are not as prevalent. Spending time learning how to identify and decode Base64 encoding will help you in the large majority of cases involving simple malware data encoding. It's quite common for malware authors to encode PE files in Base64. Since PE files contain the ASCII characters `mz` as part of the header, be on the lookout for the encoded version of `mz`, which begins with `tv`.

Data Hashing

Hashing is essentially one-way data encoding, meaning that, theoretically, it can't be reversed. When data is run through a hashing algorithm, the resulting hash will always be the same length. For example, the MD5 hashing algorithm always produces a fixed-length hash value of 128 bits, or 32 characters. SHA-1 produces a hash value of 160 bits, or 40 characters. And finally, SHA256 produces a hash value of 256 bits, or 64 characters.

Let's use CyberChef, mentioned earlier, to see what data hashing looks like in practice. In [Figure 16-2](#), I've used CyberChef and MD5 to hash a simple URL, generating the 32-character hash value shown in the Output box.

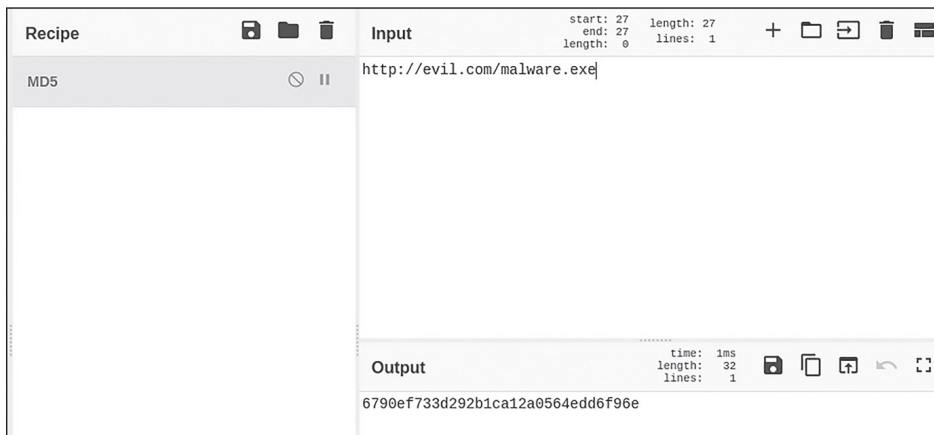


Figure 16-2: MD5 hashing with CyberChef

In [Figure 16-3](#), I've modified one character in the URL.

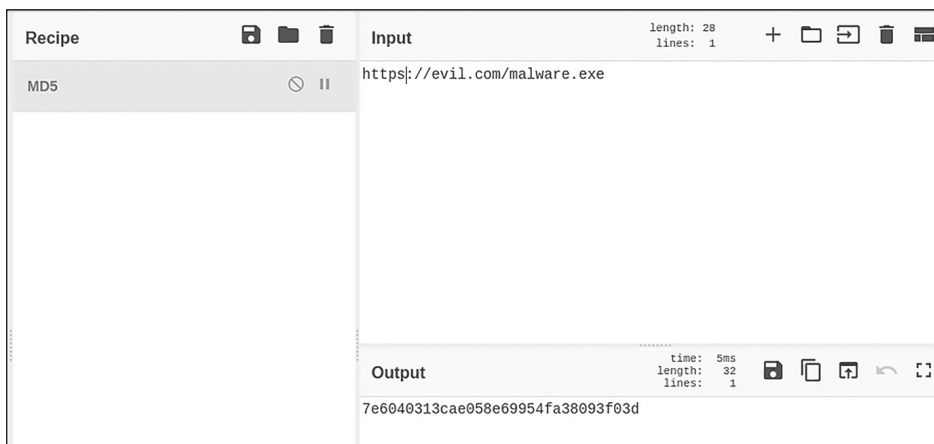


Figure 16-3: Altering the input URL in CyberChef

You may be able to spot that I simply added an `s` to the `http` string. Notice that the hash output completely changed. This effect, in which a drastically different hash value is produced for different input data (even if the input data is 99 percent identical), is known as *cascading*. Cascading divides the input data into groups of characters and then subdivides those groups.

NOTE

We'll be using CyberChef in various examples throughout the rest of this chapter. It's a great all-purpose tool for data manipulation tasks, and it has many uses outside of hashing data. It's a good idea to familiarize yourself with its features.

Malware can also take advantage of the power of hashing for obfuscation purposes, and there are a couple of different ways it can implement these

hashing algorithms within its code. One method is using the native CryptoAPI in Windows. This service provides a simple-to-use API for software (malware, in our case) to encode, decode, encrypt, and decrypt data. I'll cover CryptoAPI later in this chapter, along with an example of how it's used in malware. A second method that malware authors might use is creating their own custom hashing algorithms. The article "Windows API Hashing in Malware," from the blog site *Red Team Experiments* (<https://www.ired.team/offensive-security/defense-evasion/windows-api-hashing-in-malware>), describes a small piece of custom code that can be used to hash API function imports to hide these function names from reversers and host defenses.

Depending on the algorithm in use, hashing can be difficult to deal with when you are reversing malware. You can try using a hash-cracking tool to brute-force the input by comparing the output of a hash function to a precomputed dictionary of known inputs and their corresponding outputs. If it's a well-known algorithm, there are several tools and resources that have prehashed common strings seen in malware. For example, OALabs wrote a tool called hashdb (<https://github.com/OALabs/hashdb>), along with its companion IDA Pro plug-in, hashdb-ida (<https://github.com/OALabs/hashdb-ida>), which is a database of common malware strings. This is a great tool for querying an unknown hash value referenced in malware and obtaining the "unhashed" string. The tool supports common hashes such as CRC-32 and ROR-13, as well as custom hash implementations seen in specific malware families such as GuLoader and Lockbit.

If the malware is using an unknown, custom hashing algorithm, the process is trickier. To understand what data the malware has hashed, you need to first understand the context around that data. You may need to make educated guesses about what the cleartext strings could be, based on surrounding data. For example, if you spot cleartext function strings such as `CreateProcess` and `VirtualAlloc`, the surrounding obfuscated strings could be related to process injection functions such as `WriteProcessMemory` or `CreateRemoteThread`. Another approach is to reverse engineer the hashing algorithm itself to obtain the original input. You would first have to locate the algorithm in the code and then analyze this code statically, using a disassembler or dynamically using a debugger. We'll touch on locating such code in the context of encryption next.

Encryption and Decryption

In addition to encoding its data, malware will often encrypt it. As opposed to hashing, encryption is not a one-way procedure. Any data that has been encrypted can later be decrypted using a decryption key. As with hashing, malware often encrypts parts of its code or data in order to obfuscate its intentions, evade defenses, and protect itself from the reverse engineer.

This section will outline some encryption algorithms malware might use. However, because malware authors can even obfuscate their own encryption methods, making it that much more difficult to determine the exact encryption algorithm being used, we'll focus more on methodology than on specific implementations of encryption algorithms. Then, we'll go over some tips for identifying encryption and decryption routines in malware, as well as guidance for how to overcome these techniques.

Symmetric and Asymmetric Encryption

There are two primary forms of encryption: symmetric and asymmetric. *Symmetric encryption* involves the use of a shared (symmetric) key between two clients. If Client 1 wishes to send data to Client 2, each client must have the encryption key since it's used to both encrypt and decrypt the data. When malware uses symmetric encryption, its encryption key is either embedded inside the malware's code or generated on the fly (using standard Windows libraries, for example). Common forms of symmetric encryption include AES, DES, Blowfish, and RC4. **Figure 16-4** illustrates how symmetric encryption and decryption work at a high level.

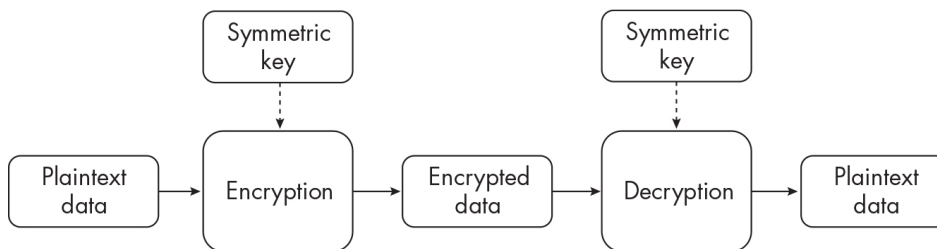


Figure 16-4: How symmetric key cryptography works

First, plaintext data is run through an encryption algorithm and encrypted with a symmetric encryption key. Once the encrypted data is ready to be decrypted, it is run through the decryption algorithm and provided the same symmetric key, which produces the original plaintext data.

There are two primary forms of symmetric encryption algorithms. *Stream ciphers* encrypt data one bit at a time and are often very fast. They're used in protocols such as SSL/TLS to encrypt web traffic. *Block ciphers* encrypt data in chunks (blocks) and typically provide stronger encryption than stream ciphers. They're also the more commonly used of the two, especially for tasks in which speed isn't a primary concern.

As opposed to symmetric encryption, *asymmetric encryption* uses two keys rather than one: a *public key* for encryption and a *private key* for decryption. If Client 1 wishes to encrypt and send data to Client 2, Client 2 must first share their public key with Client 1, which uses that key to encrypt the data. Client 2 then uses their own private key to decrypt the data. Conversely, when Client 2 wishes to encrypt data for Client 1, they encrypt the data with Client 1's public key, and Client 1 decrypts the data with their own private key. This negates the need for secure key exchange, as public keys are meant to be shared and exchanged, while private keys remain secret.

Figure 16-5 illustrates asymmetric encryption and decryption.

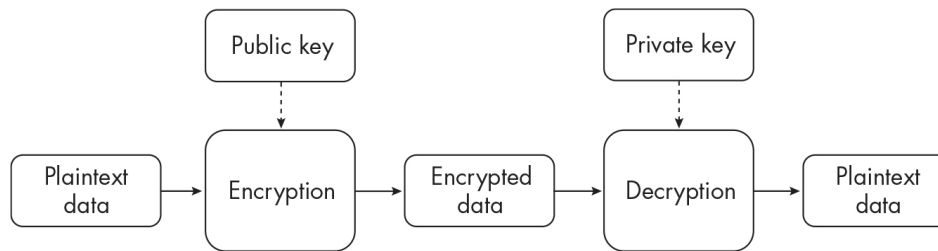


Figure 16-5: How asymmetric key cryptography works

As in **Figure 16-4**, plaintext data is run through an encryption algorithm, but this time it's an asymmetric one. The public key is used to encrypt the data, and the private key is responsible for decrypting the data back into the original plaintext version.

Malware can encrypt its code or data in order to hide it from reverse engineers or network- and host-based defenses. Encryption is also used in ransomware, a type of malware that infects its victim, encrypts selected files on the host system, and then holds those files for ransom. Upon payment, the threat actor sends the decryption key to the victim to decrypt the files. Modern ransomware uses both symmetric and asymmetric encryption methods or a hybrid approach.

Table 16-2 lists some of the common encryption algorithms used in malware.

Table 16-2: Cryptographic Algorithms Seen in Malware

Name	Type	Description
Advanced Encryption Standard (AES)	Symmetric (block cipher)	AES is considered one of the strongest forms of symmetric encryption. It is often used in ransomware.
Rivest Cipher 4 (RC4)	Symmetric (stream cipher)	RC4 is a fast and easy-to-implement algorithm, but it's not especially strong. It's used by various malware families for quick and simple data encryption and decryption.
Rivest, Shamir, and Adleman (RSA)	Asymmetric	Named after its creators, RSA is a popular algorithm that has been used by various ransomware families. It is known for its relatively high speed and efficiency.
Elliptical Curve Cryptography (ECC)	Asymmetric	ECC is a newer algorithm and has an advantage over RSA in that it is more secure for a given key size and thus more efficient. ECC is seeing increased use in malware.

Before we go into detail on a few of the most common applications of encryption algorithms in malware, we need to go over an important component of encryption: XOR.

Exclusive Or

Many encryption algorithms revolve around the *exclusive or (XOR)* operation, so it's important to understand how it works. In an XOR operation, the input data is compared bit by bit to a provided key. **Table 16-3** shows

an example where the bits in Input A and Input B are compared to produce an output value.

Table 16-3: XOR Binary Output

Input A	Input B	XOR output
0	0	0
1	0	1
0	1	1
1	1	0

As you can see, if the bits in Input A and Input B are equal, the resulting output value is 0. If the bits are not equal, the output is 1.

Data is XOR'd based on a key, which is sometimes 1 byte long (but this can be adjusted, as you'll see soon). Take a look at the XOR operation in [Table 16-4](#), which uses the 1-byte XOR key 0x35.

Table 16-4: XOR Output with the Key 0x35

Input text	h	t	t	p	s
Input (in binary)	01101000	01110100	01110100	01110000	01110011
XOR key (35)	00110101	00110101	00110101	00110101	00110101
Output (in binary)	01011101	01000001	01000001	01000101	01000110

Input text	h	t	t	p	s
Output text	]	A	A	E	F

In the Input Text row, the ASCII text string `https` is being XOR'd using the key `0x35`. The Input (in Binary) row shows the binary equivalent of this `https` string. The XOR Key (35) row shows the XOR key (`0x35`) in binary (`00110101`). The Output (in Binary) row is the XOR'd binary data, that is, the data after the XOR operation completes. And finally, the Output Text row shows the ASCII representation of the output. Note that XOR is reversible, meaning that to decrypt the output text, you simply need to XOR it again using the same key of `0x35`, which will produce a result of `https`.

This is just a simple example using a 1-byte XOR key, but in reality, malware often complicates the encryption process by using a longer key. If, for example, malware uses a 5-byte key such as `0x356C2DA155`, as shown in [Table 16-5](#), each byte in the key is XOR'd against each byte in the input data, starting with the first byte and repeating.

Table 16-5: XOR Output with the Key 0x356C2DA155

Input text	h	t	t	p	s
Input (in binary)	01101000	01110100	01110100	01110000	01110011
XOR key	00110101 (35)	11011000 (6C)	10110100 (2D)	10100001 (A1)	10101010 (55)
Output (in binary)	01011101	10101100	11000000	11010001	11011001

Input text	h	t	t	p	s
Output text	] .	Y	Ñ	&	

Now the `https` ASCII text string is XOR'd with a key of `0x356C2DA155`, which is broken down into multiple columns for simplicity. Each byte of the key will be XOR'd against one of the text bytes. The output in binary is shown in the fourth row, followed by the text representation of the XOR'd output. For a more complete explanation of the technical intricacies of XOR operations, see https://en.wikipedia.org/wiki/Exclusive_or.

XOR can be quite useful for malware authors; it is fast, efficient, and simple to implement. It does have its drawbacks, however. As I mentioned previously, you can decrypt XOR'd data by simply providing it with the same key used to encrypt it. For XOR to work properly, the XOR key must be hardcoded in the malware's code, and an experienced reverse engineer may be able to locate it. Additionally, due to the fact that XOR keys are sometimes only a byte or a few bytes long, you can often brute-force the XOR'd data by running it through all combinations of XOR keys.

There are several tools that do this. For example, XORSearch by Didier Stevens (<https://blog.didierstevens.com/programs/xorsearch/>) will search for strings in the input file, try different XOR keys (as well as other encodings), and print the results. I ran an executable through XORSearch and received the result shown in **Figure 16-6**.

```
remnux@remnux:/media/sf/vmshare$ xorsearch -p -s 569322cffc27f37cb2d6300e53a7241b
Found XOR 00 position 00000000: 000000E0 .....!..L.!This program cannot be r
Found XOR 00 position 0000EFB0: 000000D8 .....!..L.!This program cannot be r
Found XOR 01 position 0000F440: 000000C8 .....!..L.!This program cannot be r
Found XOR 01 position 000115B0: 000000C8 .....!..L.!This program cannot be r
Found XOR 05 position 0000C2B0: 000000D8 .....!..L.!This program cannot be r
Found XOR 0D position 0000050C: 000000E8 .....!..L.!This program cannot be r
```

Figure 16-6: Output from the XORSearch tool

The command in the first line includes a couple of parameters: `-p` tells XORSearch to search specifically for encoded PE files, and the `-s` switch tells XORSearch to dump the executable if it finds one. This executable has five additional executables embedded inside it, four of which are encoded with XOR (the first result can be ignored, as it simply contains the PE header of the input file). XORSearch shows the XOR key (highlighted in

column 1) and the offset address where the embedded file sits (highlighted in column 2).

You can easily spot XOR in malware using a disassembler or debugger; the instruction is simply `xor`. In IDA Pro, once your malware sample is loaded in, navigate to **Search ▶ Text**, type `xor` as the search string, and click **Find All Occurrences**. [Figure 16-7](#) shows some example output.

sub_409521	xor	al, al
sub_41DDC0	xor	al, [edx]
sub_434AF8	xor	al, [edx+0B4h]
sub_421010	xor	al, 1

Figure 16-7: Viewing `xor` instructions in IDA Pro

The first XOR operation in this figure isn't of interest to us. XORing a register with itself simply clears the register, setting it to 0. This is the most common use of XOR in executable files, and it's not specifically relevant to encryption.

The bottom three `xor` operations are the ones we care about. In these examples, a CPU register is being XOR'd with a memory address. In the `xor al, [edx]` instruction, `al` is the register and `[edx]` is the memory address. In a case like this, the register often contains the XOR key, which is being XOR'd with data stored in memory. The opposite can also be true.

If you spot XORing in code, you can decrypt that data by locating the target encrypted data and the XOR key in code. In the case of `xor al, [edx]`, you can inspect the `al` register to look for the XOR key. Additionally, inspecting the memory pointer in EDX will likely lead you to the data that is being XOR'd. Analyzing the malware dynamically inside a debugger can help with this.

Once you locate the data and XOR key, you can copy the data from the file and XOR it using the key. Any number of tools will work for this, but we'll use CyberChef again. [Figure 16-8](#) demonstrates its use with an XOR key of 1A2D3F.

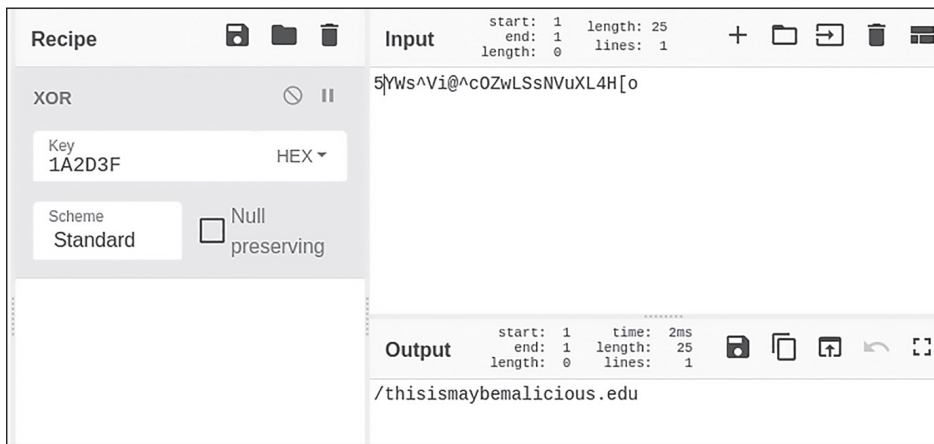


Figure 16-8: XOR decoding in CyberChef

Malware often decrypts XOR'd data and code in memory prior to executing this code. One reverse engineering tactic is to let the malware decrypt this data for you and then obtain it. You can do so using a debugger, which I'll discuss in "Decrypting Encrypted Malware Data" on [page 339](#).

As a side note, in addition to the `xor` instruction, there are a few other basic methods that some malware uses to obfuscate code and data, such as `ror` (rotate right) and `rol` (rotate left), which simply rotate the bytes a specific number of spaces in either direction. Consider, for example, the instruction `ror eax, 5`. If EAX contains the value 12345678, when you rotate all digits five places to the right, the new value is 45678123. All digits have shifted five spaces to the right so that the digits that were "pushed off" the end (4, 5, 6, 7, and 8) are now at the beginning of the string. The `rol` instruction reverses this operation.

Rivest Cipher 4

Rivest Cipher 4, or RC4, is one of the most common stream encryption algorithms used by malware because it is simple to implement and relatively strong (compared with basic encoding like Base64 or even XOR). The basic steps of how it works are illustrated in [Figure 16-9](#).

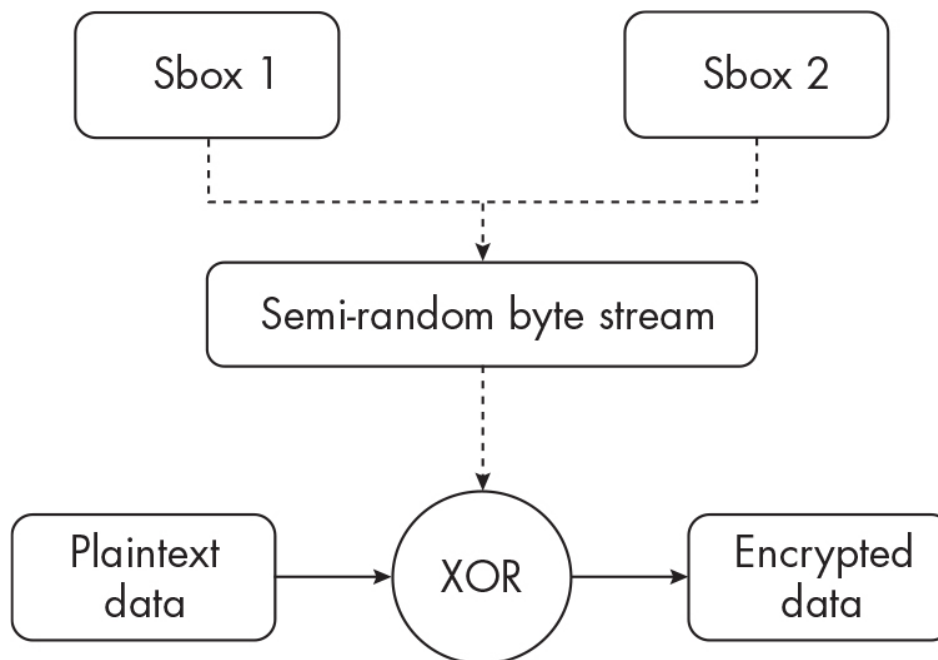


Figure 16-9: An overview of how RC4 works

First, the algorithm creates an array of values, called an *Sbox*, of 256 bytes (or 0x100 in hex). This is Sbox 1. Another Sbox (Sbox 2) is initialized and contains the encryption key. The RC4 key length can be between 1 and 256 bytes. This key can be hardcoded in the malware's executable file or generated dynamically.

Sbox 1 and Sbox 2 then combine, and their data is scrambled multiple times to create a semi-random byte stream. The plaintext (the data being encrypted) is then XOR'd with this byte stream, which is essentially an XOR key. When this encrypted data needs to be decrypted, the data is XOR'd again with the same key stream.

When dealing with encryption and decryption routines in code, you may be able to easily spot certain algorithms in use. The 256-byte array is a telltale sign that RC4 is in use. You'll likely see 0x100 referenced multiple times in the code, followed by loops, as in this code:

```

--snip--
loc_45B5F9:
--snip--
mov [eax], ebx
inc ebx
add eax, 4
❶ cmp ebx, 100h
❷ jnz short loc_45B5F9
  
```

```
loc_45B62A:
--snip--
xor eax, eax
mov al, [ebp+var_D]
mov [ebp+edi*4+var_418], eax
inc ebx
add esi, 4
③ cmp ebx, 100h
④ jnz short loc_45B62A
```

This example shows what RC4 might look like in a malware sample. The first block of code shows the Sbox 1 array being initialized and a comparison operation to `0x100` ❶. Notice the jump back to the beginning of the function ❷, signifying a loop. In the second block, there is another reference to `0x100` (Sbox 2) and the subsequent comparison operation ❸, as well as a loop ❹. These two function loops in code can often reveal that RC4 is being used.

Windows CryptoAPI

Windows provides its own implementation of many common encryption algorithms, called the *Microsoft Cryptography API (CryptoAPI)*. This API includes support for many common symmetric and asymmetric encryption algorithms, along with other services such as digital certificates and hashing. The CryptoAPI also includes an update that appeared in Windows Vista called the *Cryptography API: Next Generation (CNG)*. CNG contains the latest cryptographic libraries provided in Windows, adding support for modern algorithms as well as for custom cryptography APIs. CNG largely replaces the CryptoAPI in newer versions of Windows. Although the CryptoAPI is deprecated, it's important to discuss it, as it's still heavily used by malware because it has a long history and is well understood.

CryptoAPI Functions

When initializing the CryptoAPI, one of the key functions that applications are required to call is `CryptAcquireContext`. This function is responsible for returning a handle to a *cryptographic service provider (CSP)*, a module that performs encryption and decryption functions and stores associated encryption keys. Since CSPs are integral to some Windows programs, they are tightly controlled. Applications don't have direct access to the internals of the CSP; everything is handled indirectly via API function calls.

Table 16-6 describes `CryptAcquireContext` and some of the other important functions exposed in the CryptoAPI.

Table 16-6: Important CryptoAPI Functions

Name	Description
<code>CryptAcquireContext</code>	Gets a handle to a specific key container that is a precursor for nearly all cryptographic operations.
<code>CryptGenKey</code>	Generates a public and private key pair for asymmetric cryptography or generates a random session key.
<code>CryptGenRandom</code>	Fills a buffer with random bytes and is sometimes used to generate keys.
<code>CryptEncrypt</code>	Encrypts data from a plaintext buffer.
<code>CryptDecrypt</code>	Decrypts data that was encrypted using <code>CryptEncrypt</code> .
<code>CryptDestroyKey</code>	Destroys a handle to a key. This is a common cleanup function.
<code>CryptReleaseContext</code>	Releases the handle to a CSP and associated key containers. This is a common cleanup function.
<code>CryptCreateHash</code>	Starts the hashing process for a stream of data and initializes a hash object that will be used to store hash data.
<code>CryptDestroyHash</code>	Destroys a hash object. This is a common cleanup function.

The most interesting functions in this table are `CryptEncrypt` and `CryptDecrypt`. `CryptEncrypt` takes many parameters, such as a handle to an encryption key (created with `CryptGenKey` Or `CryptImportKey`) and a pointer to the buffer containing the plaintext data to be encrypted. If you're analyzing a malware sample that uses cryptographic operations and calls

`CryptEncrypt` to encrypt data, you can set a breakpoint on this function in a debugger and inspect the plaintext buffer, which will expose the data being encrypted! You'll see an example of this later in this chapter.

Oftentimes, developers add wrappers around these cryptographic functions to simplify implementation so that the software developer doesn't have to call each of these functions directly, which reduces programming errors. This is noteworthy because some malware authors invoke the crypto libraries directly in their code, rather than using the many wrappers that are available. Since malware developers are, in fact, human and prone to making mistakes, invoking these Windows crypto functions directly can be problematic for inexperienced malware developers. One ransomware malware family, CryptoLocker, had a flaw in its implementation of the RSA encryption algorithm, making it easy to decrypt its encrypted ransomed files. Researchers from Emsisoft discovered that CryptoLocker was implementing `CryptAcquireContext` incorrectly. One of this function's parameters accepts flag values that enable additional features. One flag, `CRYPT_VERIFYCONTEXT`, must be set, or the generated private encryption key will be stored on the local host. This flaw was quite problematic for CryptoLocker. If the decryption key is stored locally on the host, a victim can easily recover their files if they know where to look. This is exactly what Emsisoft did, and it recovered many victims' files. See the post "CryptoDefense: The Story of Insecure Ransomware Keys and Self-Serving Bloggers" on the Emsisoft blog (<https://www.emsisoft.com/en/blog/6032/cryptodefense-the-story-of-insecure-ransomware-keys-and-self-serving-bloggers/>) for more details.

The Windows CryptoAPI is fairly complex and features many different functions. For much more detailed information on these functionalities, refer to Microsoft's excellent article "CryptoAPI System Architecture" on MSDN (<https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptoapi-system-architecture>).

CryptoAPI Analysis in Practice

Now let's see how the Windows CryptoAPI looks in practice; specifically, let's examine how a hashing algorithm is implemented. This malware sample will get the DNS domain name that the victim system is part of, hash it, and compare the hash to a hardcoded hash: the domain name that the malware is targeting. If the hashes don't match (meaning the victim domain doesn't match the target domain), the malware won't infect the victim system. Additionally, storing the hardcoded domain name in a

hashed state makes it a lot more difficult (if not impossible) for a reverse engineer to discover which domain name the malware is actually looking for. Let's take a closer look in practice with the following pseudocode:

```

hashedTargetDomain = "4b557a3281181193f1b1fae7228314e77d174fa13b59f606c5400409f13875a2"; ❶

GetComputerNameExA(ComputerNameDnsDomain, domainName, dataSize); ❷

CryptAcquireContextA(&hCSP, 0, 0, 0x18, 0xf0000000); ❸
CryptCreateHash(hCSP, 0x800c, 0, 0, &hHash); ❹
CryptHashData(hHash, domainName, dataLength, 0); ❺
CryptGetHashParam(hHash, HP_HASHSIZE, hashSize, &dataLength, 0); ❻
CryptGetHashParam(hHash, HP_HASHVAL, hashedDomainName, hashSize, 0); ❼

if (hHash != 0) { ❽
    CryptDestroyHash(hHash);
}

if (hCSP != 0) { ❾
    CryptReleaseContext(hCSP, 0);
}

memcmp(hashedDomainName, hashedTargetDomain, 0x20); ❿

```

In this example, the malware first defines its target domain name, *victimcompany.com*. The SHA256 hash of this string is 4b557a3281181193f1b1fae7228314e77d174fa13b59f606c5400409f13875a2, which is stored in the variable `hashedTargetDomain` ❶. Next, the malware calls `GetComputerNameExA` with the parameter `ComputerNameDnsDomain` to get the infected victim's domain, which will be stored in the memory buffer called `domainName` ❷. Then, it calls `CryptAcquireContextA` ❸ with the following parameters: a pointer to the location in memory where the CSP handle will be stored, the key container name (in this case, `NULL`), the CSP to be used (`NULL` signifies that the default Windows provider will be used), the provider type (`0x18`, or the Microsoft Enhanced RSA and AES Cryptographic Provider), and the flags parameter. Most of these parameters are inconsequential here. The most important one to mention is the cryptographic provider type: in this case, the Microsoft Enhanced RSA and AES Cryptographic Provider, which supports a variety of hashing and encryption formats.

NOTE

For the complete list of types Microsoft supports, see “Cryptographic Provider Types” at <https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptographic-provider-types>.

After the malware gets a handle to the key container, it then calls `CryptCreateHash` to create a handle to a hash object ④. This hash object must be created prior to generating a hash. This function takes the following parameters: a handle to the newly created cryptographic service provider (hCSP), the algorithm ID (x800C, or `CALG_SHA_256`), the hash key, an optional flags value, and a pointer to the handle of the new hash object (hHash). Since the malware is passing in the value of `CALG_SHA_256` for the algorithm ID parameter, we can assume that the malware is using the SHA256 hashing algorithm.

To finally obtain the hash of the victim’s domain, the malware calls `CryptHashData` to create a hash from the domain string ⑤. When the malware calls the `CryptHashData` function, it passes in the following parameters: the handle to the hash object created previously (hHash), a pointer to the buffer that contains the data to be hashed (domainName), the data length of the hash, and an optional flags value. After this function is called, the hashed value of the domain name will be stored in the hash object.

To retrieve the hashed data, the malware must call `CryptGetHashParam` ⑥. The parameters of `CryptGetHashParam` are as follows: a handle to the hash object (hHash), the query type (the type of data being requested, such as the hash size or the hash value), a pointer to the buffer that will receive the requested data, the size of the returned data, and an optional flags value. This malware calls `CryptGetHashParam` twice: once to get the size of the hashed data, and once to get the actual hashed data (the hash of the victim’s domain name). Toward the end of this function, the malware calls `CryptDestroyHash` to destroy the hash object ⑦ and `CryptReleaseContext` to release the handle of the key container and CSP ⑧. These are standard cleanup functions.

Finally, the malware must compare the hardcoded hashed value with the hashed value of the victim’s domain name to determine if the victim is its correct target. To do this, the malware calls `memcmp`, or memory compare, a function that compares two values in memory ⑨. The values being compared are `hashedDomainName` and `hashedTargetDomain`. If these values match (meaning that the host is part of the malware’s target domain), the malware continues to infect the host. If they do not match, the malware will terminate itself.

This malware code example uses a *guardrail*, a safety measure put in place by the malware author to avoid infecting unintended victims. This type of guardrail can also be used as a sandbox evasion technique, as the malware won’t execute properly in a sandbox environment that doesn’t have the domain name the malware is looking for.

Windows Cryptographic API: Next Generation

Given that it’s simply an update to the CryptoAPI, CNG is functionally very similar. However, nearly all CNG functions are prefixed with a `B`, which helps differentiate the two APIs. For example, `CryptEncrypt` is part of the CryptoAPI, while `BCryptEncrypt` is part of the CNG. [Table 16-7](#) outlines some of the common CNG functions you may spot in malware.

Table 16-7: Important CNG Functions

Function	Description
<code>BCryptOpenAlgorithmProvider</code>	Initializes a cryptographic provider. This is very similar to <code>CryptAcquireContext</code> from the CryptoAPI.
<code>BCryptGenerateKeyPair</code>	Initializes a public/private key pair for asymmetric cryptography.
<code>BCryptGenerateSymmetricKey</code>	Generates a key for symmetric cryptography.
<code>BCryptEncrypt</code>	Encrypts plaintext data.
<code>BCryptDecrypt</code>	Decrypts data.
<code>BCryptDestroyKey</code>	Destroys a key object. This is a common cleanup function.

Because of its similarities to the CryptoAPI, I won’t cover the CNG in more detail in this book. For more information, see the “Cryptography API: Next Generation” page in Microsoft’s knowledge database (<https://learn.microsoft.com/en-us/windows/win32/seccng/cng-portal>).

Now we'll turn to some tips you can apply to overcome encryption in malware.

Practical Tips for Overcoming Encryption in Malware

When faced with a malware sample that is using cryptography to obfuscate its strings, code, or network communications, you often need to identify where these cryptographic operations are occurring in order to reveal the malware's intentions. This section will provide some general and practical advice on how to locate and identify cryptographic functions in malware code and perhaps uncover data that the malware author doesn't want you to see.

Locating and Identifying Cryptographic Routines

Generally, there are a few ways to locate and identify encryption algorithms in malware. First, you can find behaviors in the code that match a particular algorithm. For example, in RC4, as noted previously, the two 256-byte (0x100) arrays and their associated loops are often a dead giveaway. You can also inspect the malware's code for common instructions used in crypto algorithms, such as `xor` and `rol`. This is a more generic approach, but it's useful for locating many types of algorithms or even custom implementations. Lastly, looking for calls to CryptoAPI or CNG functions, such as `CryptEncrypt`, can be revealing as well.

There are several tools that can help you immensely with locating and identifying crypto routines. CAPA, which was introduced in [Chapter 3](#), can perform basic code analysis of a binary file and provide a lot of useful information that can help guide your manual code analysis of a malware sample. CAPA can also locate crypto algorithms in use in malware, as shown in [Figure 16-10](#).

```
reference public RSA key
namespace data-manipulation/encryption/rsa
author moritz.raabe@fireeye.com
scope function
mbc Cryptography::Encryption Key [C0028]
examples b7b5e1253710d8927cbe07d52d2d2e10:0x417DF0
function @ 0x13F4E6018
or:
bytes: 06 02 00 00 00 A4 00 00 52 53 41 31 @ 0x13F4E6114
```

Figure 16-10: Viewing RSA usage in CAPA

Here you can see that this sample, which happens to be a variant of the Ryuk ransomware family, is using RSA asymmetric encryption. CAPA even shows us where this data is located in the code (function 0x13F4E6018).

Likewise, in [Figure 16-11](#), CAPA located RC4 encryption in a different sample.

```

encrypt data using RC4 via WinAPI
namespace data-manipulation/encryption/rc4
author moritz.raabe@fireeye.com
scope function
att&ack Defense Evasion::Obfuscated Files or Information [T1027]
mbc Defense Evasion::Obfuscated Files or Information::Encryption-Standard Algorithm [E1027.m05]
examples 2A584DFC657348D164274A12BFF98BD8:0x404D42, 32BB43F8847ECF158C1E96891ED9A28C:0x10003A88
function @ 0x66B22CD8
and:
or:
  number: 0x6801 = CALG_RC4 @ 0x66B22D63
or:
  api: CryptDeriveKey @ 0x66B22D6C
optional:
or:
  number: 0x1 = PROV_RSA_FULL @ 0x66B22CFE, 0x66B22D84
  api: CryptAcquireContext @ 0x66B22D88
  api: CryptDecrypt @ 0x66B22D8C

```

Figure 16-11: Viewing RC4 usage in CAPA

In this extract, CAPA has identified encryption code in the malware sample. Specifically, it has discovered code possibly related to RC4, as well as several associated CryptoAPI calls (CryptDeriveKey, CryptAcquireContext, and CryptDecrypt).

IDA Pro and Ghidra disassemblers both have plug-ins available for locating and identifying crypto routines. While CAPA is a more agnostic tool that doesn't require a disassembler, the advantage of using disassembler plug-ins is that you can quickly examine the code that contains the interesting crypto functions.

Here are two plug-ins for IDA Pro:

- FindCrypt2 (<https://hex-rays.com/blog/findcrypt2/>)
- IDA Signsrch (https://github.com/nihilus/IDA_Signsrch)

And here are two for Ghidra:

- FindCrypt (<https://github.com/d3v1l401/FindCrypt-Ghidra>)
- FindCrypt (a more recent, updated version; <https://github.com/TorgoTorgo/ghidra-findcrypt>)

[Figure 16-12](#) shows the first Ghidra FindCrypt plug-in in action.

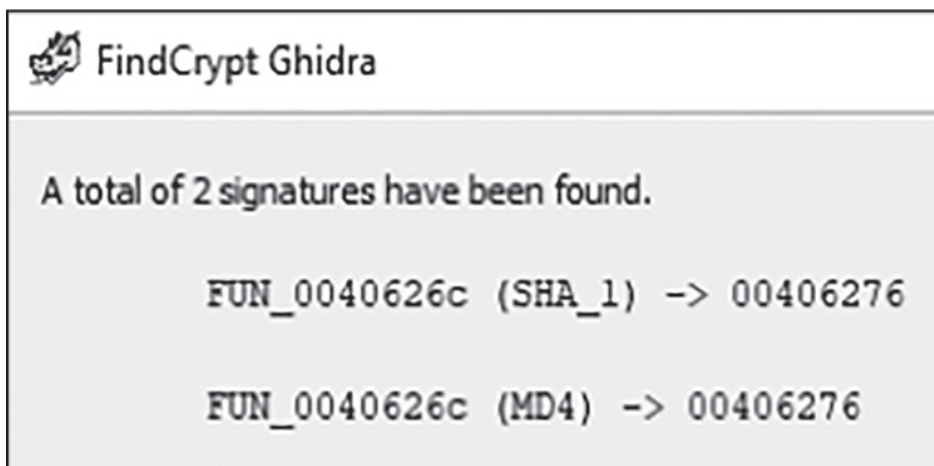


Figure 16-12: Locating encryption code with the FindCrypt Ghidra plug-in

FindCrypt has found two possible hashing algorithms in use (SHA_1 and MD4) and prints the offset to the code where the data might reside. Keep in mind that open source tools like these are always in flux; developers may stop maintaining them at any time. It's best to continually look for new tools and plug-ins to aid you in your malware analysis efforts.

Sometimes a malware sample may be using a custom crypto algorithm or an otherwise heavily modified, obfuscated, or uncommon algorithm. This is where generic identification of encryption/decryption routines is more helpful. These cryptographic routines typically follow a similar pattern.

First, the data to be encrypted or decrypted will be loaded, usually in the form of a `mov` operation. The following malware code shows data (`ebp+encrypted_data`) being moved into a register (`ebx`):

```
sub_decryptData:  
mov ebx, [ebp+encrypted_data]
```

Next, computations are performed on the data. This almost always involves loops in the code, possibly mathematical instructions (such as `add`, `sub`, `mul`, `imul`, and `div`), and `xor` or shift instructions (`shl`, `shr`, and so on):

```
xor ebx, [ebp+xor_key]
```

Finally, the manipulated data is stored for later use:

```
❶ mov [ebp+decrypted_data], ebx  
dec ecx
```

```
❷ cmp ecx, 100  
❸ jnz sub_decryptData
```

In this case, the malware is moving the newly decrypted data to the stack
❶. There will also likely be one or more loops and a loop counter ❷,
which will jump back to the data read instruction and load more data for
decryption ❸.

Now, what do you do when you've found the encrypted data?

Decrypting Encrypted Malware Data

Let's say you've located and possibly identified a crypto algorithm in a malware sample; the malware contains encrypted data that will be decrypted using this algorithm at runtime. There are two methods to approach this: static and dynamic.

Static Decryption

Static decryption allows you to decrypt the target data in the malware executable file itself, without running the malware. Static decryption has the advantage that you can run decryption tools on malware files at scale, saving you a lot of effort if you're investigating many samples at a time. The challenge of static decryption is that you must reverse engineer the crypto routines in the malware, a process that can range from easy to very difficult depending on the encryption algorithm being used and the way it is implemented. Malware authors also can change encryption keys or algorithms across different samples of the malware, which may negate the "at scale" advantage just mentioned.

To conduct static decryption, you must first identify the encryption algorithm in place and the encryption keys required, which are often hard-coded in the malware or residing in memory. The Python API PyCrypto (<https://pypi.org/project/pycrypto/>) can be used to automate tasks like this. Alternatively, CyberChef is a great tool for decoding many different data types and can help you quickly decrypt the data you find in malware.

Dynamic Decryption

Dynamic decryption involves running the malware (or emulating the code in the malware), allowing the malware to decrypt its secrets, and "catch-

ing” the decrypted data using the help of a tool like a debugger. The advantage of dynamic decryption is that it is often less time-consuming than static decryption methods and a great way to quickly get to the data you want. The downsides are that dynamic decryption is more difficult to do at scale and that malware often guards its encrypted data with anti-analysis traps.

Let’s walk through two analysis scenarios that will teach you some tricks for quickly decrypting malware’s secrets dynamically. These are general tricks that work independently of the encryption algorithm employed by the malware.

Decrypted Code and Data Capture

Malware may contain encrypted code that is dynamically decrypted in memory at runtime. This helps the malware evade host-based defenses and makes reverse engineering difficult. When malware employs these techniques, the most efficient way to identify the data being decrypted is to catch it in its decrypted state! In the next example, this malware sample decrypts shellcode in memory and executes it. Let’s see if we can capture this shellcode after it is decrypted.

NOTE

This example uses a malware executable that you can download from VirusTotal or MalShare, using the following file hash:

SHA256:
db136799d9e0854fdb99b453a0ebc4c2a83f3fc823c3095574832b6ee5f96708

First, load the sample into your disassembler of choice. I’ve used IDA Pro. To identify the encryption algorithm used in this sample and locate the decryption code, you can use a tool such as CAPA, or you can simply search for xor operations using the disassembler’s search functionalities.

Figure 16-13 shows part of the output of this XOR search in IDA Pro.

CODE:0045B724	sub_45B5AC	xor	eax, eax
CODE:0045B733	sub_45B5AC	xor	edx, edx
CODE:0045B752	sub_45B5AC	xor	[esi], al
CODE:0045B7B1	sub_45B794	xor	eax, eax
CODE:0045B862	sub_45B794	xor	ecx, ecx

Figure 16-13: Viewing xor instructions in IDA Pro

There are many instances of the `xor` instruction in this executable, most of them benign. Remember, XORing a register with the same register essentially clears that register. So, `xor eax, eax`, `xor ecx, ecx`, and `xor edx, edx` aren't noteworthy to us. Let's check out the instructions `xor [esi], al` by double-clicking the entry. [Figure 16-14](#) shows the result.

CODE:0045B735	push	edx
CODE:0045B736	push	eax
CODE:0045B737	mov	eax, [ebp+ebx*4+var_418]
CODE:0045B73E	add	eax, [ebp+edi*4+var_418]
CODE:0045B745	cdq	
CODE:0045B746	call	sub_405268
CODE:0045B748	mov	al, byte ptr [ebp+eax*4+var_418]
CODE:0045B752	xor	[esi], al
CODE:0045B754	inc	esi
CODE:0045B755	dec	[ebp+var_18]
CODE:0045B758	jnz	loc_45B6A2

Figure 16-14: Viewing the encryption routine in IDA Pro

This code block contains several mathematical instructions, such as `xor` and `add`. There are also several `mov` instructions that appear to be shifting data around, as well as `inc`, `dec`, and `jnz`, indicating a loop. At first glance, this appears to be a crypto function!

If you inspect the code blocks above the function we're currently in, you may spot some additional indicators of cryptography in use. There's a `cmp ebx, 100h` instruction and what appears to be another loop, as shown in [Figure 16-15](#).

CODE:0045B609	nop	
CODE:0045B60A	mov	[eax], ebx
CODE:0045B60C	inc	ebx
CODE:0045B60D	add	eax, 4
CODE:0045B610	cmp	ebx, 100h
CODE:0045B616	jnz	short loc_45B5F9

Figure 16-15: Viewing an RC4 loop in IDA Pro

Additionally, at offset `0x0045B684`, there's another loop and `cmp ebx, 100h` instruction, shown in [Figure 16-16](#).

CODE:0045B676	mov	al, [ebp+var_D]
CODE:0045B679	mov	[ebp+edi*4+var_418], eax
CODE:0045B680	inc	ebx
CODE:0045B681	add	esi, 4
CODE:0045B684	cmp	ebx, 100h
CODE:0045B68A	jnz	short loc_45B62A

Figure 16-16: Another RC4 loop in IDA Pro

This is looking like RC4. Now that we've likely found the encryption routine in use, we can identify where in the code the data will be fully decrypted, set a breakpoint on this address in a debugger, and wait for the data to be decrypted for us! First, we should identify where to set our breakpoint. The function we're currently in is `sub_45B794`. If you select this function in the IDA Pro disassembler or graph view and press X, you'll find a list of cross-references to it (that is, other functions that are calling `sub_45B794`). Double-click the function that appears in the list.

You should see the instruction `lea ebx, [ebp+var_BEEP]`, as indicated here:

```
--snip--
call sub_45B5AC
call GetConsoleCP
lea edx, [ebp+var_BEEP]
--snip--
```

This `lea` instruction is loading an address into the EDX register. This is interesting because it comes directly after the RC4 decryption function we were just investigating (`sub_45B5AC`). The address of this `lea` instruction is `0045B850`. This is a good target for our debugger breakpoint.

For the debugging, I'll use x64dbg. You could also use the built-in debugger in IDA Pro or another debugger of your choice. Load the sample into the debugger and select **Run to User Code** (to jump to the start of the malware's code), set a breakpoint on the code of interest (**bp 0045B850**), and then run it (press F9 on your keyboard). This particular sample will sleep for 10 seconds or so before executing the code of interest (see [Figure 16-17](#)).

•	0045B846	E8 61FDFFFF	call evil2.45B5AC
•	0045B848	E8 60AFFAFF	call <JMP.&GetConsoleCP>
→	0045B850	8D95 1541FFFF	lea edx,dword ptr ss:[ebp-BEEP]
•	0045B856	B9 E4650000	mov ecx,65E4

Figure 16-17: Setting a breakpoint on the decrypted code in x64dbg

Once the breakpoint is hit, right-click `[ebp-BEEP]` and select **Follow in Dump ▸ Address: EBP-BEEP**. As [Figure 16-18](#) shows, we should now see our decrypted data in the Dump window!

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	[x=] Locals	Struct
Address	Hex	ASCII					
0018402D	78 78 78 78	78 78 78 78	78 78 78 78	78 78 78 78	78 78 78 78	xxxxxxxxxxxxxxxx	
0018403D	88 45 08 88	40 34 55 E9	67 61 00 00	78 6A 68 66		.E..@4Uéga..xjhf	
0018404D	89 95 34 FF	FF FF 5A 6A	72 66 89 95	36 FF FF FF		..4yyyZjrf..6yyy	
0018405D	5A E9 25 06	00 00 78 78	78 78 78 78	78 78 78 78		Zé%...xxxxxxxxxx	
0018406D	78 78 78 78	78 78 78 58	6A 74 66 89	45 C8 58 E9		xxxxxxxxxjtf.EÈxé	
0018407D	02 29 00 00	78 78 78 78	78 78 78 78	78 78 78 78		.)...xxxxxxxxxxxx	
0018408D	78 78 78 78	78 78 78 89	5D 88 E8 AF	38 00 00 81		xxxxxxx.]è;...;	
0018409D	EC 76 45 00	00 81 C4 0E	01 00 00 81	EC 1A 11 00		ivE...À....i...	
001840AD	00 81 EC 1E	78 00 00 E9	02 04 00 00	78 83 F8 01		..i.{...é....x.ø.	
001840BD	0F 85 30 37	00 00 8D 8D	28 F7 FF FF	33 D2 E9 76		..07....(÷yy30év	
001840CD	06 00 00 78	78 78 78 78	78 78 78 78	78 0F 82 70		...xxxxxxxxxxx..p	
001840DD	00 00 00 5E	8D 7D E4 E9	3D 4D 00 00	E9 F5 18 00		...^}âé=M..éö...	
001840ED	00 78 78 78	78 78 78 78	78 78 78 78	78 78 78 78		..xxxxxxxxxxxxxxxx	
001840FD	78 78 78 78	78 78 78 8B	DA 6A 49 E9	6B 14 00 00		xxxxxxx.ÚjIék...	
0018410D	78 78 78 78	78 78 78 6A	75 58 E9 27	22 00 00 78		xxxxxxxxxjuxé'".x	
0018411D	78 78 78 78	78 78 78 78	78 78 78 78	78 78 8D 44		xxxxxxxxxxxxxxxx.D	
0018412D	24 10 50 E8	E9 18 00 00	59 38 C7 0F	85 06 00 00		\$.Pèé...Y;Ç.....	
0018413D	00 6A 00 FF	54 24 1C 53	E9 D3 3B 00	00 78 78 78		.j.yT\$.Sé0;...xxx	
0018414D	78 78 56 8D	8C 55 C8 FD	FF FF 0F 87	01 8D 70 BF		xxV...UèVyy...Dz	

Figure 16-18: Viewing the decrypted code dumped in x64dbg

This data is unfortunately not easily human readable. Let's inspect it further. Extract this data by right-clicking the Dump window and selecting **Follow in Memory Map**. Then, right-click the highlighted memory region (it will be highlighted in gray in the Memory Map window) and select **Dump Memory to File**.

If you run the `strings` command on the file or use a PE strings utility like PESTudio, you'll see a few strings, but not much of interest. However, by using the strings deobfuscator tool FLOSS (which you might remember from [Chapter 2](#)), we can deobfuscate some of the data. You can run the FLOSS tool like so:

```
> floss --format sc32 shellcode_dump.bin
```

Here we are telling FLOSS to treat this file as 32-bit shellcode. You can see output from FLOSS, which has recovered 52 stack strings:

```
FLOSS extracted 52 stackstrings
vbox
Set WshShell = CreateObject("WScript.Shell")
HARDWARE\DEVICEMAP\Scsi\Scsi Port 0\Scsi Bus 0\Target Id 0\...
Ident
advapi32
VboxGuest.sys
vmware
virus
--snip--
sandbox
sample
--snip--
```

It appears that this malware is building strings dynamically on the stack to obfuscate its data. You might be able to spot several suspicious strings in this output; note the references to sandboxes and virtual machines. This code is actually shellcode that the malware decrypts and then runs to conduct some basic sandbox checks. We won't get into these details, however; this example is intended only to show the value of dynamic decryption. This trick not only works for RC4 but also for many other algorithms.

Capturing Code and Data Before Encryption

Malware often employs encryption techniques to obfuscate network traffic, such as communication with a C2 server. The malware sample shown in [Figure 16-19](#) calls `BCryptEncrypt` to encrypt its C2 information before sending the data to its control infrastructure. To capture the data in clear-text, I simply set a breakpoint on `BCryptEncrypt`.

74181958	90	nop	
7418195C	8BFF	mov edi,edi	BCryptEncrypt
7418195E	55	push ebp	
7418195F	8BEC	mov ebp,esp	
74181961	83EC 14	sub esp,14	
74181964	A1 1C401C74	mov eax,dword ptr ds:[741C401C]	

Figure 16-19: Setting a breakpoint on `BCryptEncrypt` in x64dbg

NOTE

You can find this sample on [MalShare](#) or [VirusTotal](#) with the following hash:

SHA256:

b2562b0805dd701621f661a43986f9ab207503d4788b655a659b61b2aa095fce

In the case of `BCryptEncrypt`, the second value on the stack is the pointer to the buffer containing the plaintext data to be encrypted. This data appears to be my VM's hostname (see [Figure 16-20](#)).

Address	Hex	ASCII
0278A2A8	51 00 48 00 67 00 45 00 49 00 42 00 46 00 5A 00	Q.K.g.E.I.B.F.Z.
0278A2B8	51 00 00 00 00 00 00 00 EF DB AE 85 00 08 00 88	Q.....i0µ...
0278A2C8	01 00 00 00 20 00 00 00 09 A6 6A 80 E8 0A 51 BA	!j'e.Q°
0278A2D8	E1 9C 4D 8D 0D C0 D7 44 A7 3D 82 67 9A 10 08 19	á.M..ÀxD\$=.g...
0278A2E8	C7 4E A5 F7 07 2D 50 83 34 00 00 00 18 00 00 00	CN%÷.-P*4.....
0278A2F8	56 49 43 54 49 4D 58 50 43 58 36 37 31 36 32 5F	VICTIMXPX67162
0278A308	35 43 34 45 43 37 43 33 B8 08 8B 76 AA 65 34 01	5C4EC7C3...v^e4.
0278A318	10 27 00 00 B9 AD 01 00 01 00 00 00 00 00 00 00	
0278A328	D2 DB 9B B5 00 0C 00 8A 68 00 74 00 74 00 70 00	00.µ....h.t.t.p.

Figure 16-20: Viewing the data to be encrypted in x64dbg

Remember that the `CryptEncrypt` and `BCryptEncrypt` functions are used to encrypt data, and `CryptDecrypt` and `BCryptDecrypt` decrypt data. These functions are very useful for malware analysts to know. Simply place a breakpoint on these functions in a debugger, let the function run, inspect the buffer containing the data, and you've saved yourself a lot of analysis effort!

Summary

This chapter discussed encoding, hashing, and encryption techniques that malware might use to obfuscate its code and data, adding one more layer of protection against analysis and reverse engineering and one more evasion technique to counter defenses. I hope you've picked up some useful tips you can use when investigating malware that employs these tactics. In the next chapter, we'll look at another form of malware obfuscation: packers.